



DATA STRUCTURES · CIS 4020

Algorithm Analysis

From counting steps to Big-O



Why not just time it?

A common instinct: run the program and see how many nanoseconds it takes.

2 ns

Execution 1

1 ns

Execution 2

2 ns

Execution 3

3 ns

Execution 4

Same program, four runs, three different readings. A stopwatch measures one run on one machine — not how the algorithm behaves as the input grows.

Imagine the fizzbuzz program: print every integer from 1 to 100. For multiples of 3, print “fizz”; for multiples of 5, print “buzz”; for multiples of 15, print “fizzbuzz.” (e.g. 3 → fizz, 5 → buzz, 15 → fizzbuzz)

But, three different programs can solve fizzbuzz correctly. Which one is “better,” and how would we know without a stopwatch?



Three candidate algorithms

Algorithm A

```
if ((i % 15) == 0)
    print("fizzbuzz")
if ((i % 3) == 0 &&
    (i % 15) != 0)
    print("fizz")
if ((i % 5) == 0 &&
    (i % 15) != 0)
    print("buzz")
```

Algorithm B

```
if ((i % 15) == 0)
    print("fizzbuzz")
else if ((i % 3) == 0)
    print("fizz")
else if ((i % 5) == 0)
    print("buzz")
```

Algorithm C

```
if ((i % 3) == 0)
    print("fizz")
if ((i % 5) == 0)
    print("buzz")
```

All three print exactly the same output for every integer from 1 to 100. What differs is how much work each one does to get there — that is what the rest of this unit measures.

STEP ONE: JUST COUNT



Counting Algorithm A (a rough estimate)

Algorithm A checks every integer against up to five conditions: $i \% 15$, then (if not already caught) $i \% 3$ and its guard, then $i \% 5$ and its guard.

The exact count actually depends on:

- Code arrangement
- The order the conditions are written in
- The order of the “and”s
- Whether “and” short-circuits
- The calls to print itself

≈ 500

100 integers $\times$ 5 checks

Total: about 500 checks



Counting Algorithm B

Algorithm B's else-if chain means only one branch ever runs per integer, so we can count exactly how many checks each group of integers costs.

Group	Count of integers	Checks each	Subtotal
Multiples of 15	6	× 1	6
Multiples of 3 (not 15)	27	× 2	54
Multiples of 5 (not 15)	14	× 3	42
Everything else	53	× 3	159

Total: 6 + 54 + 42 + 159 = 261 checks

STEP THREE: THE SIMPLEST COUNT



Counting Algorithm C

Algorithm C has no guard conditions and nothing to chain: every integer is checked exactly twice, once against 3 and once against 5, no matter the outcome.

200

100 integers $\times$ 2 checks, total 200 checks

Three counts so far

≈ 500

Algorithm A

261

Algorithm B

200

Algorithm C



What exactly counts as one step?

However carefully we count checks, counting comparisons is still not quite the full picture. Look at just one branch:

```
if ((i % 15) == 0)
    print("fizzbuzz")
```

Modulus

Comparison

Print

This one line is really at least three steps — and every branch in Algorithm B contains them. From here on, we analyze algorithms by their execution steps, not by lines of code.



A harder example: the multiplication table

Write a program that prints a formatted multiplication table from a minimum to a maximum value. For example, from -3 to 1 :

×	-3	-2	-1	0	1
-3	9	6	3	0	-3
-2	6	4	2	0	-2
-1	3	2	1	0	-1
0	0	0	0	0	0
1	-3	-2	-1	0	1

This time, the amount of work depends on two nested loops, not one pass through a list — so we need a new way to count.



Counting the nested loop

```
01 int result = 0;
02 for (j = min; j <= max; j++) {
03     print(j + " ");
04     for (i = min; i <= max; i++) {
05         result = j * i;
06         print(result + " ");
07     }
08     println();
09 }
```

Rough estimate. We could probably break down each line into even more steps.

Let $n = \text{max} - \text{min} + 1$, the number of rows (and columns).

Line		How many times
01	result = 0	1 time
02	j = min	1 time
	j <= max	n + 1 times
	j++	n times
03	print(j + " ")	n times
04	i = min	1 time, per outer iter.
	i <= max	n + 1 times, per outer iter.
	i++	n times, per outer iter.
05	result = j * i	n times, per outer iter.
06		n times, per outer iter.
08	println();	n times



From counts to a formula

Line(s)	Count
01	1
02 (outer loop)	$1 + (n+1) + n = 2n + 2$
03	n
04-06 (inner loop, per outer iter)	$(1 + (n+1) + n + n + n) \times n = (4n + 2)n = 4n^2 + 2n$
08	n

Total steps: $1 + (2n + 2) + n + (4n^2 + 2n) + n = 4n^2 + 6n + 3$

Size matters — the step count depends on n , the “problem size,” not on some fixed number of operations.



A thought experiment

Using $4n^2 + 6n + 3$, with n as the multiplication table's size:

n	$4n^2$	$6n$	3	Total
1	4	6	3	13
2	16	12	3	31
3	36	18	3	57
4	64	24	3	91
5	100	30	3	133
6	144	36	3	183

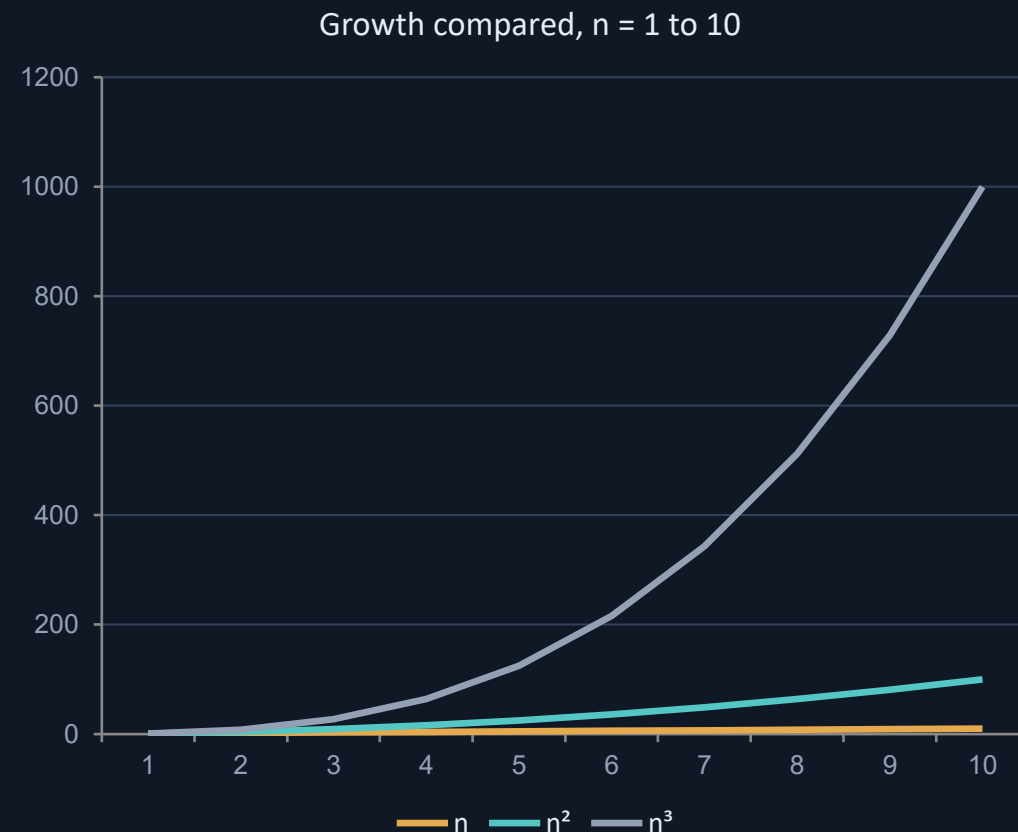
- Which term becomes the greatest factor as n grows?
- What does that term represent in the algorithm?



From loops to growth rates

- A single loop over n items: $O(n)$.
- Two nested loops over n items: $O(n^2)$.
- Three nested loops: $O(n^3)$.
- What matters most is the fastest-growing term, not the exact formula.

Notation	Name
$O(1)$	constant
$O(\log_2 n)$	logarithmic
$O(n)$	linear
$O(n \log_2 n)$	loglinear (linearithmic)
$O(n^2)$	quadratic
$O(n^3)$	cubic
$O(2^n)$	exponential





Problem size vs. estimated steps

Function	n=10	100	1,000	10,000	100,000	1,000,000
$O(1)$	1	1	1	1	1	1
$O(\log_2 n)$	3	6	9	13	16	19
$O(n)$	10	10^2	10^3	10^4	10^5	10^6
$O(n \log_2 n)$	30	664	9,965	10^5	10^6	10^7
$O(n^2)$	10^2	10^4	10^6	10^8	10^{10}	10^{12}
$O(n^3)$	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
$O(2^n)$	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

Doubling the problem size barely moves an $O(1)$ or $O(\log n)$ algorithm, but it is catastrophic for an $O(2^n)$ one — this is the whole reason growth rate matters more than a single measurement.



Rules of Big-O

- Drop lower-order terms — they stop mattering once n is large.
- Drop constant multiplicative factors — they do not change how fast something grows.
- When unsure, usually estimate up.
- You can add growth rates: $O(f(n)) + O(g(n)) = O(f(n) + g(n))$

Back to our own formula: $4n^2 + 6n + 3$. Drop the lower-order terms ($6n$ and 3): $4n^2$. Drop the constant factor (4): n^2 . So the multiplication table program is $O(n^2)$.

IT DEPENDS ON THE INPUT



Best, worst, and average case

Imagine an integer sorting algorithm. The same input size can mean very different amounts of work.

Best case

The data is already sorted. This does not happen often and we tend to ignore it, but we shouldn't.

Average case

The items are uniformly, randomly distributed. This is the more typical situation in practice.

Worst case

The items start in reverse order. This may not be common, but it will happen. Often the best judge of an algorithm.

These case scenarios are not the same for every sorting algorithm.



Sequential search

Begin with the first item, look at each one in turn, and stop as soon as you find it (or reach the end).

Case	Big-O	Why
Worst	$O(n)$	It's the last item – or not present at all
Average	$O(n)$	Really less than a full n , but still grows with n
Best	$O(1)$	It's the first item*

***The best case only describes one special situation here. Cases vary widely depending on the algorithm. We focus on the worst and average cases because those are often the most realistic and interesting.**



Binary search: cutting the problem in half



Each step throws away half of what is left — k such steps to reach a single item, think $n/2^k = 1$. Thus, k is a factor of performance.

Assume n is a power of two

$n/2^k = 1$, therefore $n = 2^k$.

Solving for k : $k = \log_2 n$.

So binary search takes $O(\log_2 n)$ steps in the worst case.

If n is not a power of two

Then n sits between two powers of two: $2^{k-1} < n < 2^k$.

Taking $\log_2$ of all three parts: $k - 1 < \log_2 n < k$.

Thus, k is always within 1 of $\log_2 n$. That constant does not change the Big-O classification.

Either way: $O(\log_2 n)$.

Splitting the input in half, again and again, often leads to $O(\log_2 n)$ or $O(n \log_2 n)$ algorithms.